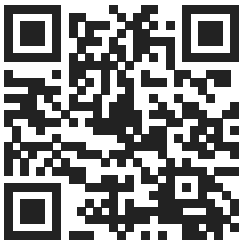
The Loop Economy

A universal combinatorial marketplace
without a platform

Peter Földiák @petfold

November 2026

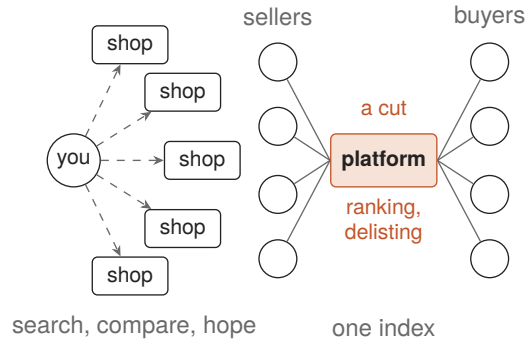
loopmarket · ontodag · factbond · on Swarm



github.com/petfold/loopmarket

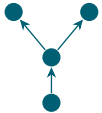
Most good trades never happen

- Markets let strangers cooperate, voluntarily: a part of individual freedom.
- Online, that means **search**: shop after shop, each with its own catalogue, terms and checkout, and you never know you found the best deal.
- Or a **platform** that owns the index: it takes a cut, decides who is seen, and can delist anyone.



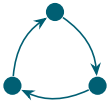
Most mutually profitable trades fail on discovery, not on price: finding them costs more than they are worth.

Describe, don’t search: solvers find the trade



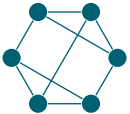
universal

Anything that can be described exactly: rides, repairs, parcels, storage, energy, care… and money, as one more offer. Not only tokens.



combinatorial

Many offers combine into one trade: loops of three, five, ten makers; a courier’s leg composed into a delivery. Algorithms do the searching.



permissionless

Every maker keeps their own offer book on Swarm. No platform, no protocol fee, nobody who can delist you.

CoW Protocol finds coincidences of wants among tokens, to beat the AMM price.
loopmarket finds them where there is no AMM to beat.

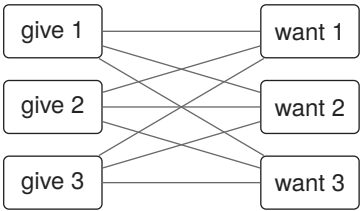
One offer form for everything

give	a signed **intent**: what, never how	a want has exactly the same form
maker	Bob	his key signs it
what	**ride**	ontodag terms
where	from 19.065, 72.865 ±1 km to 19.09, 72.87 ±2 km	boxes: GPS ± tolerance
when	6 November 2026, 18:00–20:00	a time window
price	5, on Bob's own scale	a private unit: only ratios count
bond	20, held by an escrow contract	his guarantee
arbitrator	0x51c2…7e09	a key both sides accept
id	sha256 of the canonical bytes	never changes; withdrawing is a new record

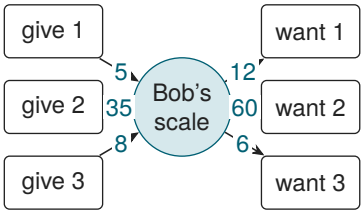
Own scale: k gives and m wants need $k + m$ prices, not $k \times m$ exchange rates.

```
$ loop give ride 'from(Jio)' 'to(BOM)' 'time(2026-11-06T12:30:00Z..2026-11-06T14:30:00Z)' 5
```

Why a personal scale



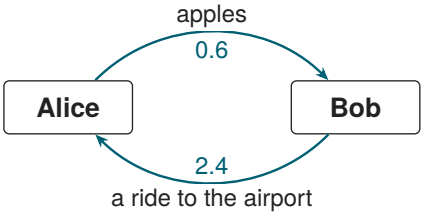
$k \times m$ exchange rates to state,
and they can contradict each other



$k + m$ prices on one private scale;
every rate is a ratio: no
arbitrage against Bob

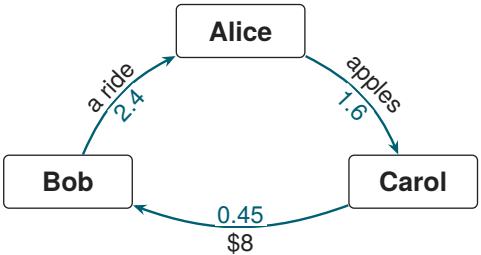
The scale is bookkeeping, a personal numeraire: nothing is issued, held or transferred. Only ratios leave the maker, and round a loop the scales cancel. A batch is scored by the sum of the logs of its loops' surplus factors, so no common unit is ever needed.

Loops: the trade nobody could find



$\prod r = 0.6 \times 2.4 = 1.44 > 1$

Barter needs a *double coincidence of wants*, which is rare.

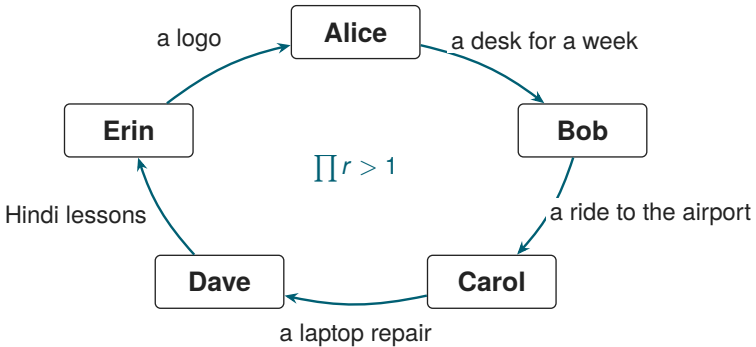


$\prod r = 1.6 \times 0.45 \times 2.4 = 1.728 > 1$

Bob wants money, not apples. No pair can trade, yet the loop clears: money is one more leg.

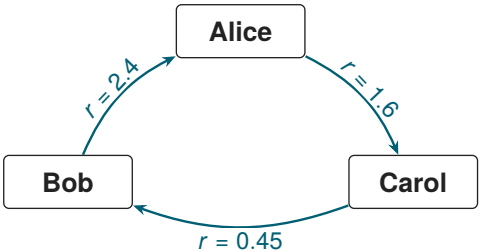
A leg's rate r is the wanter's price over the giver's, each on their own scale. $\prod r > 1$ is a **negative cycle** under $-\log r$: Bellman–Ford finds it, as in DEX arbitrage, and the makers keep the surplus.

A loop of five: no pair, no three can trade

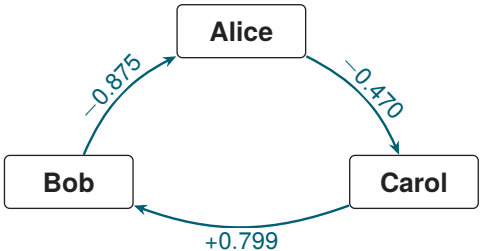


Each wants what only the one before has: Bob a desk, Carol a ride, Dave a repair, Erin lessons, Alice a logo. Nobody finds this by searching; for a solver it is one more cycle in the graph of matches. Clearing checks all five legs, and commits them together or not at all.

A profitable loop is a negative cycle



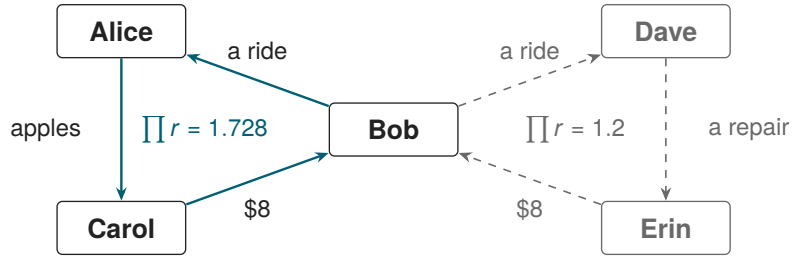
$\prod r = 1.6 \times 0.45 \times 2.4 = 1.728 > 1$



$\sum(-\log r) = -0.547 < 0$

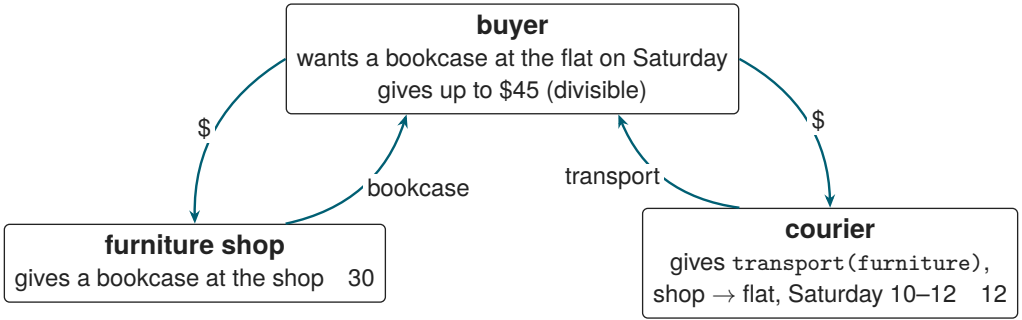
- Weigh each leg by $-\log r$: a loop with $\prod r > 1$ becomes a cycle whose weights sum below zero, which Bellman–Ford finds in $O(V \cdot E)$: the standard arbitrage detector.
- The baseline search is deterministic (U6) and may round; clearing re-checks the product in exact rationals (U9), so rounding never clears a loop.

Many candidate loops, one offer: selection



- Bob has one ride and one want. Both loops need them, so at most one of the two can clear.
- Selection packs the candidates: the set of loops worth most within every offer's capacity, scored by $\sum \log \prod r$, a number with no unit.
- Exact branch and bound while the set is small, a deterministic greedy beyond it, and one total order for ties: every replica picks the same set.

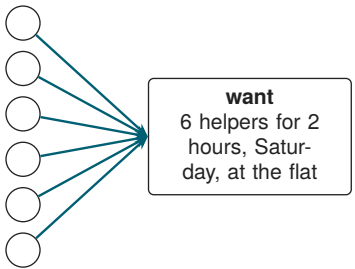
One want, many gives: a circulation



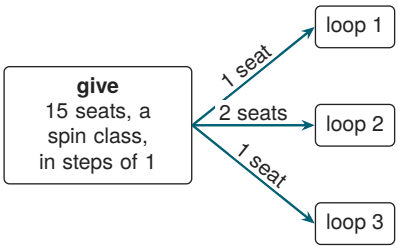
bookcase at the shop $\otimes$ `transport(furniture)` $\sqsubseteq$ bookcase at the flat

A circulation, not a cycle: every maker gives and receives, and nets to zero on their own scale. The *one* want holds the conjunction, so it clears all or nothing, and clearing splits the one payment. A bookcase is furniture; a piano is not.

Quantities: aggregation and capacity

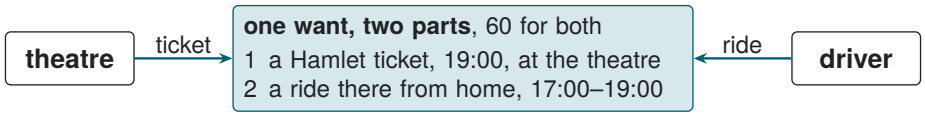


Aggregation. Six neighbours each give one helper. The want takes all six or none, and its one payment splits six ways.



Capacity. Each loop takes what its want asks; the fills never sum past 15 (U11). A floor, `8seat..15seat:1`, runs only for at least eight in one loop.

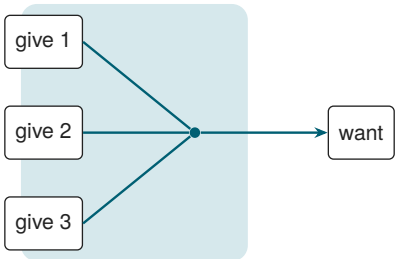
Parts: a want of several things, all or nothing




```
$ loop want theatre-ticket hamlet venue + ride 'from(home)' 'to(venue)' 60
```

- The ticket is worthless without the ride, and the ride without the ticket. Only you care, so the coupling lives in your want: its parts clear together or not at all.
- You price the bundle once; clearing splits it across the gives.
- Bundles live only on the want side: no give is ever conditional on another give.

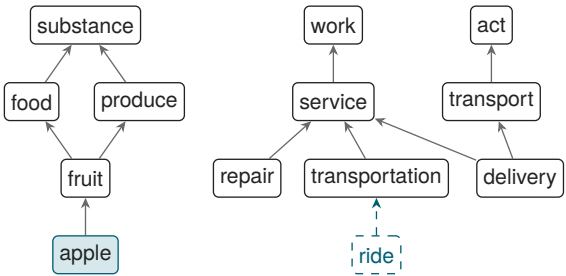
The object: a circulation on a hypergraph



one hyperedge: all or nothing

- A leg is an edge between makers. A composed or aggregated leg is a **hyperedge**: several gives, one want, one flow, filled together or not at all.
- A cleared set is a **circulation**: at every maker, value in equals value out, on that maker's own scale. A loop is its smallest case.
- Clearing prices are **node potentials**: each maker's scale is one. At those prices the rates round every cycle multiply to 1: Kirchhoff's voltage law, in logs.
- So checking a proposal is a potential check, linear in the number of legs: something an on-chain verifier can afford.

OntoDAG: categories form a DAG



edges from ontodag's core pack (a selection); ride is added with `put`

- **One relation, *fits within*.** An arrow points to the wider category; everything below a category is a special case of it.
- **Several parents are normal.** An apple is food *and* produce; a jigsaw is a saw *and* a power tool; a delivery is a service *and* a transport. No folder has to be chosen.
- **Keep no link you can infer.** The stored graph is minimal, so equal content gives an equal graph.

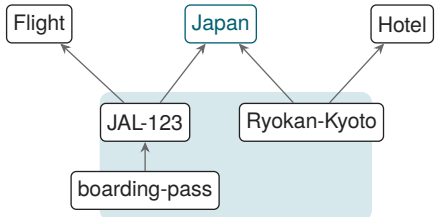
put and get: a query is an intersection of cones

```
$ odag put Flight; odag put Japan
$ odag put Hotel
$ odag put JAL-123 Flight Japan
$ odag put Ryokan-Kyoto Hotel Japan
$ odag put boarding-pass JAL-123

$ odag get Japan
JAL-123  Ryokan-Kyoto  boarding-pass

$ odag get Flight Japan
JAL-123  boarding-pass

$ odag get food produce (with the core pack)
almond apple apricot artichoke ... 117 items
```



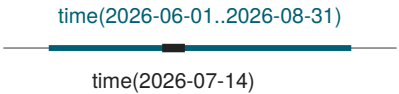
the shaded cone is everything below Japan

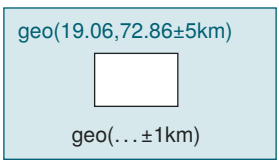
- `put` files a name under its parents; `get` returns what lies below every name you give. The boarding pass was never filed under Japan: it is below the flight, which is.
- In loopmarket a want's terms are one `get`; a give is one of its answers.

Typed values: time, place, mass are categories too

```
$ odag put JAL-123 'time(2026-07-14)'
$ odag get Flight 'time(2026-06-01..2026-08-31)'
JAL-123  boarding-pass
```

`time(2026-11-06)`	a day: the set of its instants
`geo(48.8584,2.2945±500m)`	a square 500 m either way
`mass(..5kg)`	at most 5 kg
`temperature(24C)`	exact, on the kelvin scale





- A value is a set; one sits below another when its set lies inside the other's. The order is computed from the names, never stored: no edge links 14 July to the summer.
- Values are exact rationals of SI units (500 g is $\frac{1}{2}$ kg), so every replica computes the same order. Overlap is a different question: it is not transitive, so it is never *fits within*.

Matching in loopmarket: three relations, one rule

want	give		why
fruit	apple	✓	meaning: the give lies within the want
apple	fruit	×	a fruit might be a banana
within 5 km of Jio	within 1 km of Jio	✓	place: one box contains the other
box A	box B, partly overlapping	×	overlap is not containment
Friday 6 November	Friday, 18:00–20:00	✓	time: one window contains the other
a bookcase, moved	`transport(furniture)`	✓	an operator's argument: the moved thing lies within it
a piano, moved	`transport(furniture)`	×	a piano is not furniture
flurb	anything	×	an unknown word never matches: it fails closed

Whoever fixes a value states a fact; whoever leaves it open states what they accept; the fact must lie within the acceptance. Your own places stay private: `loop place home 46.05,14.50+-5km` publishes only the box.

A shared catalogue, pinned by every offer

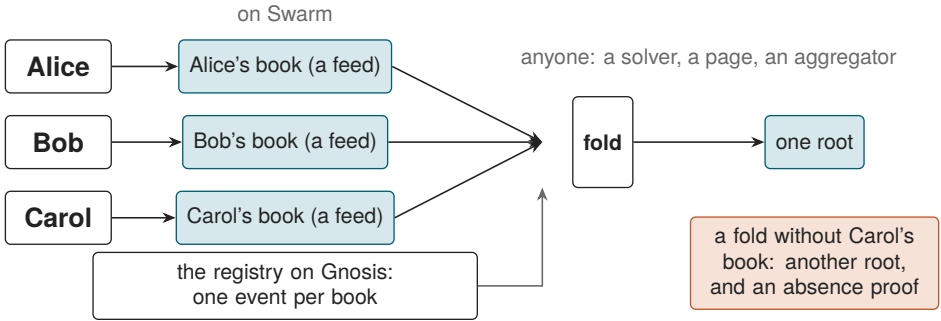
- **Canonical.** Equal content gives an equal root hash, whoever builds it and in whatever order.
- **Pinned.** Every offer names the catalogue root it was written against. Offers pinned to different roots never pair, so a match is reproducible forever.
- **Verifiable.** An answer that one category lies below another can carry a certificate that anyone checks against the root.
- **A shared core.** About 4,800 categories in fourteen branches, built by consensus over WordNet, SUMO, OpenCyc, schema.org, YAGO, BFO, DOLCE and Wikidata: each edge carried by two independent sources or an explicit ruling. Ten domain packs add about 6,700 more.
- **On Swarm.** The packs are published there and adopted by fingerprint; adopting one reproduces its root byte for byte.
- **In a browser.** ontodag's own organiser page runs the package in the browser, and is itself a Swarm site.

Swarm: a server nobody owns

what Swarm gives	what loopmarket does with it
Content addressing: data is named by its hash	an offer's id is its hash; equal books give equal roots, byte for byte; inclusion proofs are checked on chain
Feeds: mutable pointers, signed by their owner	each maker's book is their own feed: identity is a key; no account, no platform
Postage: storage prepaid, no hosting account; it all goes to the node operators	a maker pays only for their own book, and reading it costs them nothing
Thousands of independent nodes, permissionless	nobody can delist an offer; an aggregator that drops a book is caught by an absence proof
Encryption and access control, client side	delivery details are sealed to the counterparty, inside the book
Websites are content too	the market page is itself a Swarm site: it folds every book in the browser

On chain only what needs consensus: one announcement per book, the batches, the escrow.

One market from many books: the fold



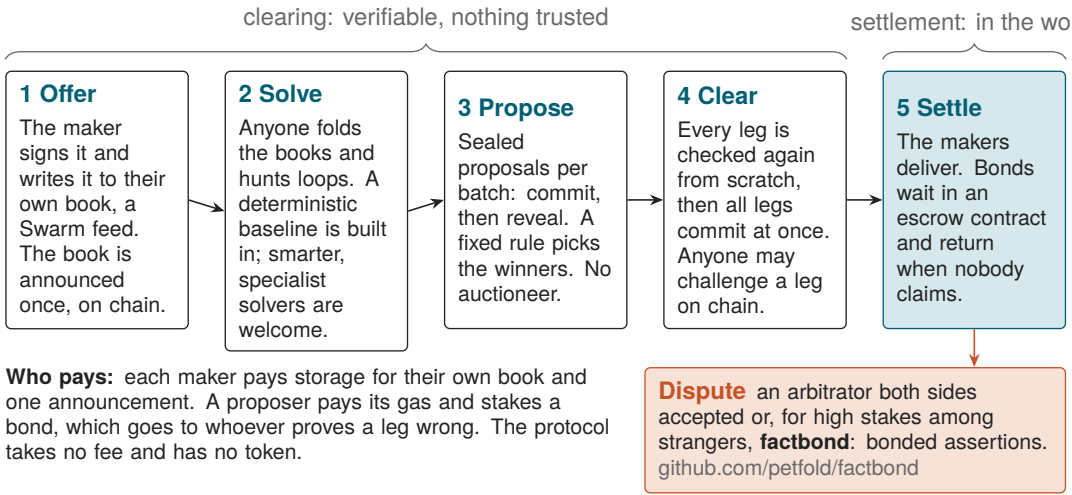
- Each maker writes only their own book and announces it once. Anyone folds the announced books, under fixed rules: forgeries refused, withdrawals kept.
- Same books, in any order: the same root. So an aggregator is only a cache.

The market in a browser: a dynamic website on Swarm

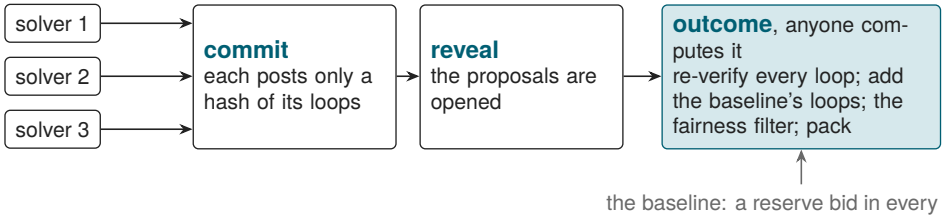


- A static page with dynamic data: the market lives in the makers' feeds. A new version of the page moves one feed, and the names stay.
- No install and no key to read; through your own Bee node it also publishes your offers. Checked in Chromium, Freedom Browser and on an iPhone.

From offer to delivery



The sealed batch: competition without an auctioneer



- Sealed: nobody can copy or front-run a proposal they cannot see. No auctioneer: every replica computes the same winners from the revealed set.
- The fairness filter (CoW's CIP-67, generalised to cycles) drops a proposal that leaves an offer worse off than its best alternative; then packing, scored in logs of surplus.
- Not built yet: threshold-encrypted sealing, and paying solvers (a spread leg).

Optimistic clearing: why anyone may challenge



- Why: the offers live on Swarm, and checking every leg on chain would cost far more than recording commitments. The chain checks only what someone disputes; whether a give fits within a want is the arbiter's.
- Who: the makers a false batch would bind, rival solvers it blocks, anyone after the bond. Open: challenges are run by hand today, and bonds sized to the damage are not built.

Settlement: delivery, in the world

- **Bonds.** A give can be backed by a bond held in an escrow contract and reserved for each fill. A want can require one (“a no-show costs me 20”), and a bond declared but not deposited matches nothing.
- **Most cases need no ruling.** A reservation returns to the giver after a quiet claim period, or at once on the wanter’s countersignature. A giver who cancels pays the amount the wanter’s ladder sets for that notice.
- **Disputes** start with a notice and a chance to cure, then go to an arbitrator both sides accepted, whose ruling is final. For high stakes among strangers there is **factbond**: bonded assertions, a ladder of rulings, deposits forfeited on reversal.
- **Credentials.** A want may require statements from registers it trusts; the counterparty gate fails closed.

github.com/petfold/factbond

Who pays for what

	pays	gets
maker	storage for their own book; one announcement	the trade, at or better than their own price
solver	the gas to post a proposal, and a bond	designed: a spread leg inside the loop it found
challenger	gas	the bond of a proposal it proves wrong
aggregator	its own storage and bandwidth	paid serving, never paid inclusion
the protocol	**nothing, either way**: no fee, no token, no rewards. A real loop looks exactly like a wash trade, so any reward tied to volume would pay people to trade with themselves.	

CoW Protocol and loopmarket

	CoW Protocol	loopmarket
traded	tokens	anything the catalogue describes, money included
matching	the same two token addresses, a limit price	fits within, in the catalogue: meaning, place, time
unit of price	token amounts, scored in the native token	each maker's own scale; scored in logs, unit-free
intents live in	CoW's order book	each maker's book, a Swarm feed
auctioneer	the autopilot, one per chain	none: sealed batches on chain, a rule anyone recomputes
solvers	bonded and allowlisted	anyone who posts the bond
after the trade	nothing: the tokens arrived	delivery, bonds in escrow, arbitration

Same family: signed intents, batches, solvers trusted with nothing. Different market: CoW's rings compete with an AMM price, a loop of rides and repairs with no trade at all. The fairness filter is CoW's CIP-67, generalised to cycles.

Where it stands: open source, live at test amounts

loopmarket
Python, BSD-3, on PyPI

✓ offers, books on Swarm, the fold, the baseline solver, clearing

✓ on Gnosis: announcements, sealed batches, challengeable clearing, escrow

✓ the `loop` command line; the market in a browser

○ next: paying solvers, privacy, first real users

ontodag
Python, BSD-3, on PyPI

✓ the DAG store, typed values (time, geo, mass, . . .), canonical roots, verifiable answers

✓ core: about 4,800 categories, plus ten domain packs, also on Swarm

factbond
bonded assertions

✓ the assertion contract on Gnosis, the escrow's resolver

○ next: the scored simulation that gates real stakes

github.com/petfold/loopmarket

github.com/petfold/ontodag

What stays hard: oracles for physical delivery · private matching (ZK fits-within proofs) · wash loops · capture of the catalogue · books thick enough to close loops

Backup: invariants the tests enforce

U1 One uniform offer form: a thing against the maker's own scale.

U2 Offers are immutable canonical values; the id is a hash.

U3 Clearing trusts no solver: every leg re-derived.

U4 Solvers work on pinned roots.

U5 Positive rates only.

U6 A deterministic baseline solver.

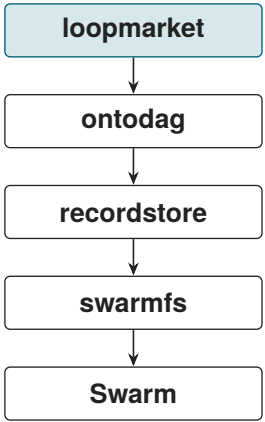
U7 Vocabulary fails closed.

U9 Exact rationals in everything clearing re-checks.

U11 No partially filled loop survives a merge unnoticed.

About 475 tests; the contracts' tests run on a local EVM every night.

Backup: the stack



offers, matching, solver, clearing, the `loop` command line

the catalogue: a DAG with typed values and canonical roots

versioned key-value books: canonical roots, merges, proofs

Swarm's chunks, feeds and signers

optional: everything also runs in memory

On Gnosis: the book registry, the beat contract with its leg verifier, the sealed batch, the escrow, and factbond's assertions as the escrow's resolver.